

I want to create a React web application using React version 16.14. I want to use webpack and JavaScript instead of Typescript.

Follow this step-by-step guide to set up a React 16.14 application using Webpack and JavaScript (without TypeScript).

Step 1: Initialize the Project

Create a new directory for your project, navigate into it, and initialize a new Node.js project: [OneUptime](#)

Bash



```
mkdir react-webpack-js-app
cd react-webpack-js-app
npm init -y
```

Step 2: Install Dependencies

Install **React 16.14.0** alongside `react-dom`, and configure your development dependencies for Webpack and Babel. [OneUptime](#)

Bash



```
# Install specific React 16.14 version
npm install react@16.14.0 react-dom@16.14.0
```

```
# Install Webpack and server tools
npm install --save-dev webpack webpack-cli webpack-dev-server html-webpack-plu

# Install Babel to transpile modern JS and JSX
npm install --save-dev @babel/core @babel/preset-env @babel/preset-react babel

# Install CSS loaders (optional, for styling)
npm install --save-dev style-loader css-loader
```

Step 3: Configure Babel

Create a file named `babel.config.js` in the root of your project directory:

JavaScript



```
// babel.config.js
module.exports = {
  presets: [
    '@babel/preset-env',
    '@babel/preset-react'
  ]
};
```

Step 4: Configure Webpack

Create a file named `webpack.config.js` in the root of your project directory: Medium

JavaScript



```
// webpack.config.js
const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');

module.exports = {
  entry: './src/index.js',
```

```

output: {
  path: path.resolve(__dirname, 'dist'),
  filename: 'bundle.js',
  clean: true,
},
module: {
  rules: [
    {
      test: /\.js$/,
      exclude: /node_modules/,
      use: {
        loader: 'babel-loader',
      },
    },
    {
      test: /\.css$/,
      use: ['style-loader', 'css-loader'],
    },
  ],
},
resolve: {
  extensions: ['.js', '.jsx'],
},
plugins: [
  new HtmlWebpackPlugin({
    template: './public/index.html',
  }),
],
devServer: {
  static: './dist',
  port: 3000,
  open: true,
  hot: true,
},
mode: 'development',
};

```

Step 5: Create Project Structure and Source Files

Set up the standard folder structure inside your project root:

Plaintext



```
react-webpack-js-app/  
├─ public/  
│   └─ index.html  
├─ src/  
│   ├── App.jsx  
│   └─ index.js  
├─ babel.config.js  
├─ webpack.config.js  
└─ package.json
```

1. public/index.html :

HTML



```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8" />  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>React 16.14 Webpack App</title>  
  </head>  
  <body>  
    <div id="root"></div>  
  </body>  
</html>
```

2. src/App.jsx :

JavaScript



```
import React from 'react';  
  
function App() {  
  return (  
    <div>
```

```
    <h1>Hello from React 16.14 with Webpack and JavaScript!</h1>
  </div>
);
}

export default App;
```

3. `src/index.js` :
(Note: React 16 uses the traditional `ReactDOM.render` API).

JavaScript



```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
import './index.css'; // Optional

ReactDOM.render(<App />, document.getElementById('root'));
```

Step 6: Add NPM Scripts

Open your `package.json` file and add the scripts to run the development server and build the production bundle: Medium

JSON



```
"scripts": {
  "start": "webpack serve --config webpack.config.js",
  "build": "webpack --config webpack.config.js"
}
```

Step 7: Run Your Application

To spin up the local development server, execute:

Bash



```
npm start
```

Your browser should automatically open to `http://localhost:3000`, displaying your React 16.14 application. To build for production, run `npm run build`.

Now I want to migrate this app to React 17

Follow this step-by-step guide to migrate your React 16.14 application to **React 17**.

Step 1: Update React Packages

Update `react` and `react-dom` to version **17.0.2** (the final and most stable release of React 17) in your terminal:

Bash



```
npm install react@17.0.2 react-dom@17.0.2
```

Step 2: Update Your Entry Point (`src/index.js`)

React 17 introduced a new JSX transform that allows you to write JSX without explicitly importing `React` in every file, but more importantly, it introduced a new root API preparation style (`ReactDOM.render` is still fully supported in React 17 without warnings, but the entry point standard remains compatible).

However, React 17 changes how `ReactDOM.render` is imported and used with the new JSX transform. Open your `src/index.js` file and ensure it is updated to use the standard React 17 import:

JavaScript



```
// src/index.js
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
import './index.css'; // Optional

ReactDOM.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
  document.getElementById('root')
);
```

Step 3: (Optional) Enable the New JSX Transform

React 17 lets you omit `import React from 'react'` at the top of files that only use JSX. To enable this in your Babel configuration, update your `@babel/preset-react` setting in `babel.config.js`:

JavaScript



```
// babel.config.js
module.exports = {
  presets: [
    '@babel/preset-env',
    [
      '@babel/preset-react',
      {
        runtime: 'automatic', // Enables the new JSX transform
      },
    ],
  ],
};
```

Note: If you enable the `automatic runtime`, you no longer need to write `import React from 'react';` at the top of component files like `App.jsx`, though keeping it won't break anything.

Step 4: Verify and Run Your Application

Clear your Webpack cache (if needed) and restart your development server to ensure everything builds correctly under React 17:

Bash



```
npm start
```

Your app is now successfully running on **React 17** with Webpack and JavaScript!

Now I want to migrate this application to React 18.

Follow this step-by-step guide to migrate your React 17 application to **React 18**.

Step 1: Update React Packages

Update `react` and `react-dom` to version **18.2.0** (or the latest stable 18.x version) in your terminal:

Bash



```
npm install react@18.2.0 react-dom@18.2.0
```

Step 2: Update Your Entry Point (`src/index.js`)

React 18 introduces a brand new root API (`ReactDOM.createRoot`). The legacy `ReactDOM.render` API is deprecated in React 18 and will trigger warnings in your console.

Open your `src/index.js` file and replace `ReactDOM.render` with `ReactDOM.createRoot` :

JavaScript



```
// src/index.js
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css'; // Optional

// 1. Get the container DOM element
const container = document.getElementById('root');

// 2. Create a root using createRoot
const root = createRoot(container);

// 3. Render your App into the root
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Step 3: Verify and Run Your Application

Restart your development server to test your application with React 18:

Bash



```
npm start
```

Your application is now successfully running on **React 18**!

Now I want to migrate this application to React version 19.

Follow this step-by-step guide to migrate your React application to **React 19**.

Bacancy Technology

Step 1: Update React Packages

Install the latest version of **React** and **React DOM** (version 19) in your terminal:

Bacancy Technology

Bash



```
npm install react@19 react-dom@19
```

Step 2: Ensure the Modern JSX Transform is Enabled

React 19 requires the modern JSX transform. Since you followed the previous step to set `runtime: 'automatic'` in your Babel config, you are already prepared. [React](#)

Verify that your `babel.config.js` matches this configuration so you don't run into transform warnings:

JavaScript



```
// babel.config.js
module.exports = {
  presets: [
    '@babel/preset-env',
    [
      '@babel/preset-react',
      {
        runtime: 'automatic',
      },
    ],
  ],
}
```

```
],  
};
```

Step 3: Review Codebase for React 19 Breaking Changes

React 19 drops support for a few legacy patterns. Check your codebase to ensure none of these are present:

`defaultProps` **on function components:** Removed in React 19. Use standard JavaScript default parameters instead. [Pennine Technolabs](#)

String Refs: Legacy string refs (e.g., `ref="myRef"`) are completely removed; use callback refs or `useRef` instead. [Bacancy Technology](#)

`useFormState` : Renamed/replaced by `useActionState` (if you are utilizing form hooks). [Bacancy Technology](#)

Step 4: Verify and Run Your Application

Restart your development server to verify that your bundle builds successfully under React 19:

Bash



```
npm start
```

Your application is now successfully running on **React 19** with Webpack and JavaScript!